



Máster en
Inteligencia
Artificial
UMU

Deep Learning para el procesamiento de Lenguaje Natural

Tema 1. Introducción al Deep Learning.
Redes Neuronales Recurrentes

JUAN A. BOTÍA BLAYA

CIENCIA DE LA COMPUTACIÓN E INTELIGENCIA ARTIFICIAL

DEPT. INGENIERÍA DE LA INFORMACIÓN Y LAS COMUNICACIONES

Universidad de Murcia

Máster en Inteligencia Artificial

2026/2027



El lenguaje es inherentemente secuencial (I)



Máster en
Inteligencia
Artificial
UMU

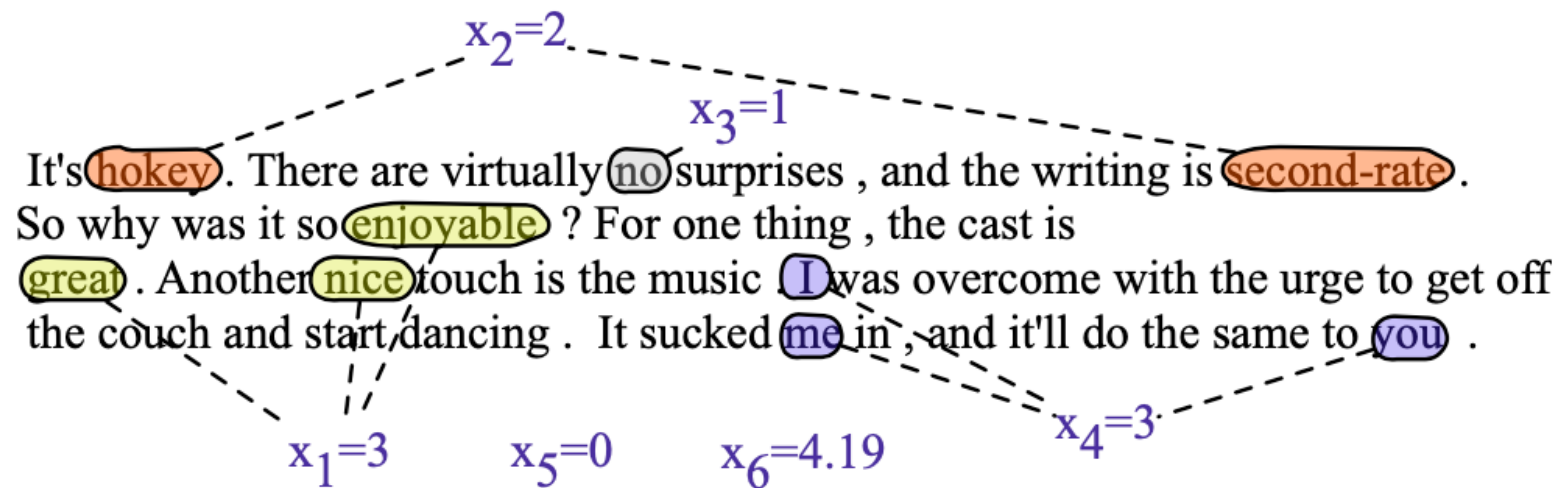
- El lenguaje es una secuencia de símbolos de entrada y salida para oyente y hablante respectivamente
- El lenguaje es una secuencia de eventos que se despliega a medida que el tiempo avanza

It's hokey . There are virtually no surprises , and the writing is second-rate .
So why was it so enjoyable ? For one thing , the cast is
great . Another nice touch is the music . I was overcome with the urge to get off
the couch and start dancing . It sucked me in , and it'll do the same to you .

El lenguaje es inherentemente secuencial (II)



Las redes neuronales que conocemos no tienen esto en cuenta
(transparencia 54 de la clase de introducción)

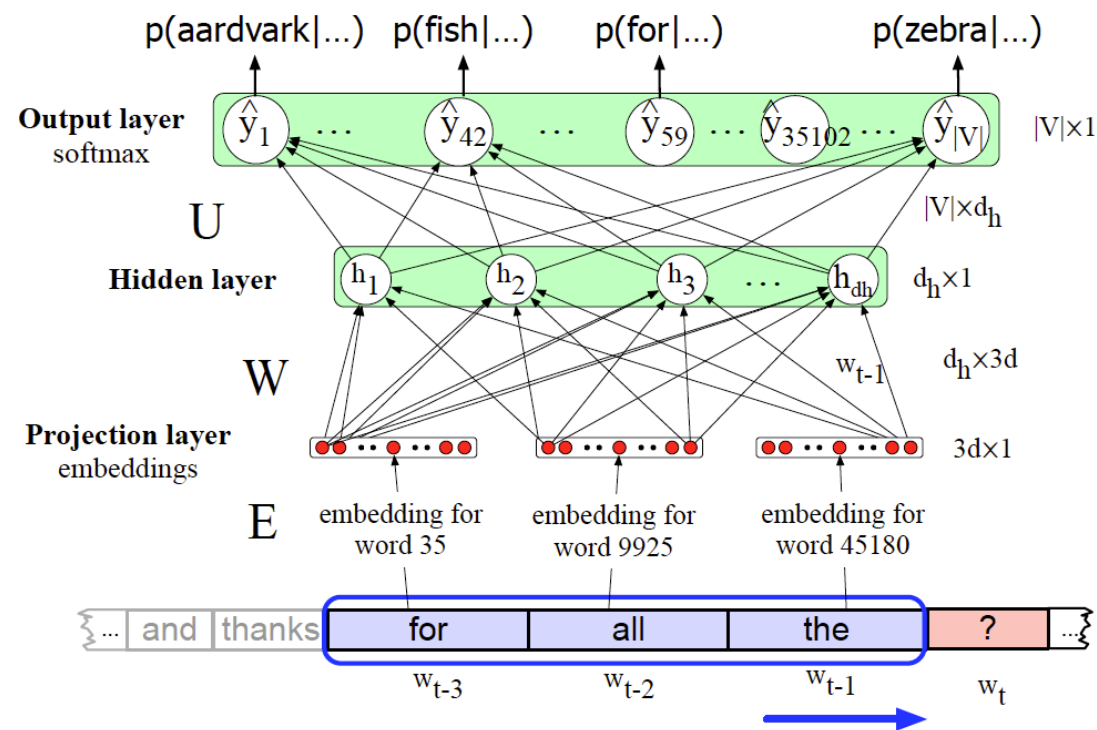


El lenguaje es inherentemente secuencial (III)

Solo lo tuvimos en cuenta una vez, con el enfoque de ventana deslizante



Máster en
Inteligencia
Artificial
UMU





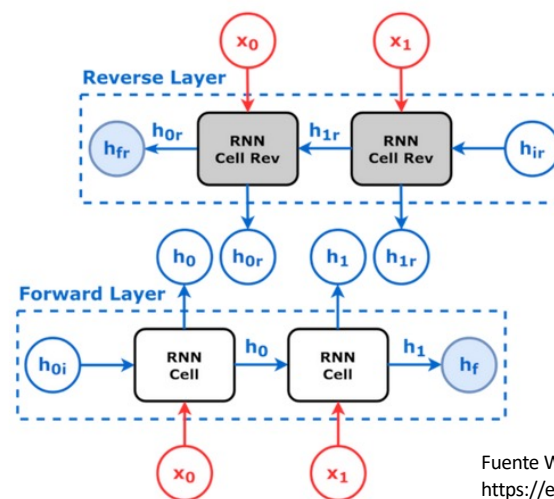
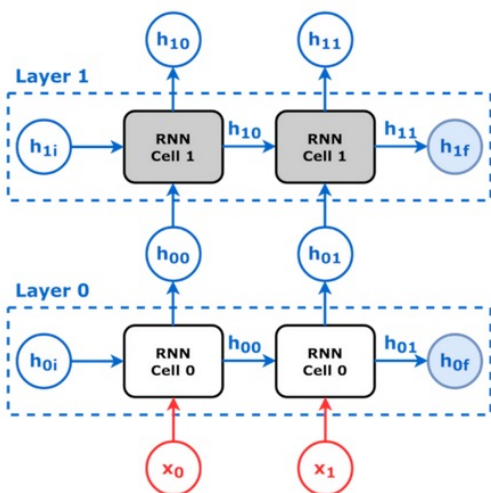
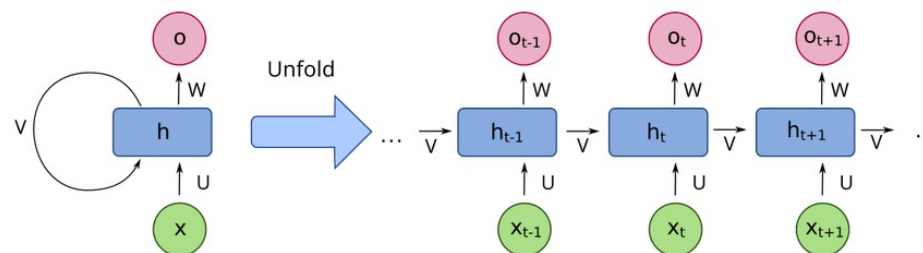
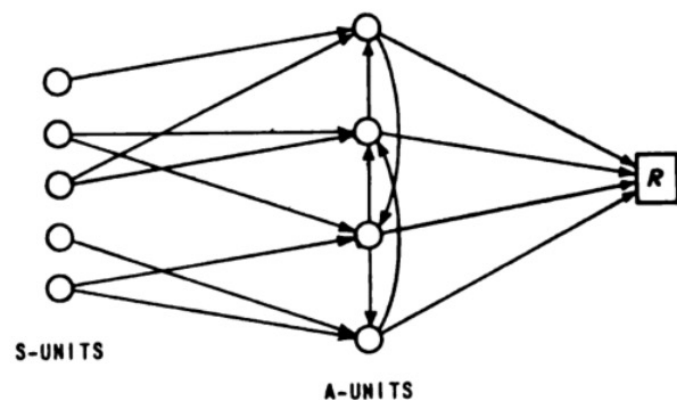
Redes recurrentes

- Definimos una red recurrente (RNN) como aquella que **contiene ciclos en sus conexiones** de red
 - El valor de salida de algunos de sus nodos depende, directa o indirectamente, de su propia salida en algún momento anterior.
- Esta definición es demasiado genérica, necesitamos restringirla para hacerla manejable
- RNNs simples: aquellas en las que en la capa oculta, la salida de sus nodos dependen de la salida de esos mismos nodos en el instante anterior

Algunos ejemplos de redes recurrentes



Máster en
Inteligencia
Artificial
UMU



Fuente Wikipedia
https://en.wikipedia.org/wiki/Recurrent_neural_network



2025/2026

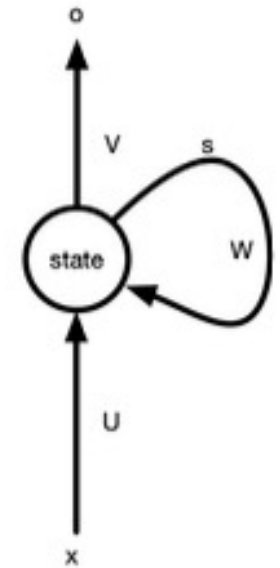
Deep Learning para el Procesamiento de Lenguaje Natural



¿Son más complejas que las redes convencionales?

En realidad, no

- Seguimos teniendo como entrada un vector de longitud fija
- Seguimos teniendo una capa oculta donde ocurre casi todo el proceso al margen de la entrada y salida
- Necesitamos ahora un nuevo conjunto de pesos, eso sí, que indica cómo se conecta la salida de la capa oculta anterior (h_{t-1}), con la actual, h_t para producir la salida en el instante t , y_t
- En la figura, *state* hace referencia a la capa oculta
 - Para referirnos al estado oculto utilizaremos indistintamente s_t y h_t (esto es $s_t = h_t$ y $s_{t-1} = h_{t-1}$)

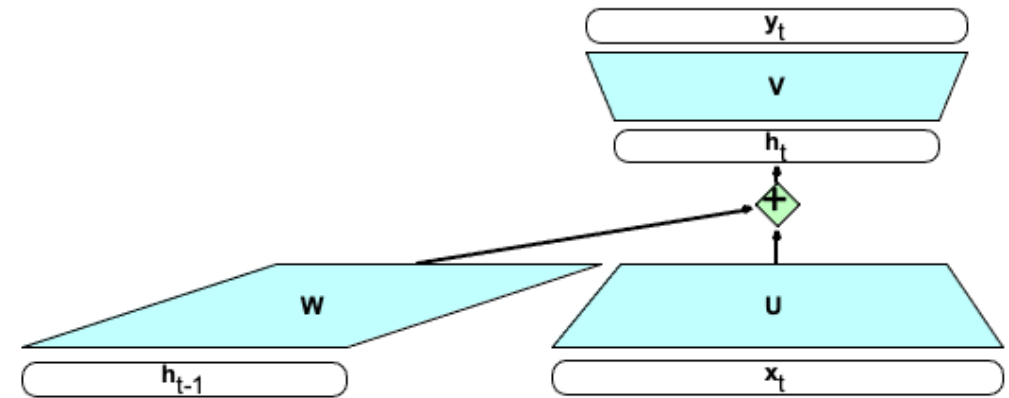




Los parámetros de una RNN

Ahora tenemos tres matrices diferentes (en azul)

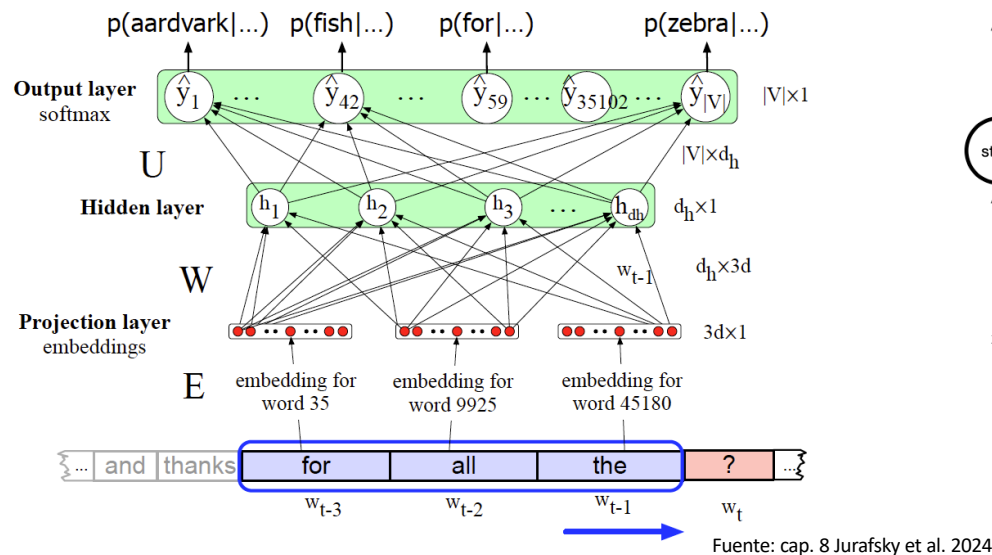
- $U \in R^{n \times d}$ parametriza cómo tratamos individualmente cada token a la entrada
- $W \in R^{d \times d}$ parametriza la recurrencia: la relación entre el estado actual s_t y el estado anterior s_{t-1} .
- $V \in R^{d \times k}$ parametriza la salida de la red: genera la salida o_t .



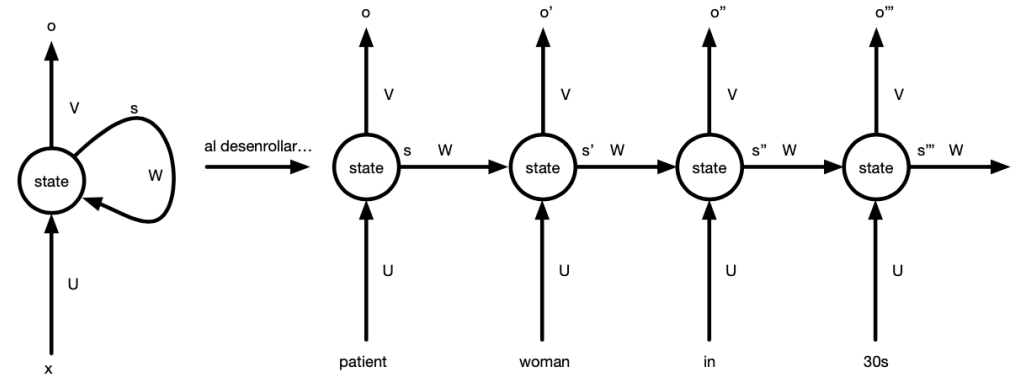
Fuente: cap. 8 Jurafsky et al. 2024



Inferencia: las RNNs comparten parámetros a lo largo de los tokens



La matriz W del MLP tiene parámetros propios para cada uno de los tres tokens de la Ventana.
NO SE COMPARTEN PARÁMETROS



La matriz W de la RNN es la misma para todos los tokens, pues los trata por separado. SE COMPARTEN LOS PARÁMETROS



Inferencia: la recurrencia se basa en la noción de estado

Si la entrada es $x = [\text{patient, woman, in, 30s}]$, la secuencia de operaciones que realiza la red recurrente para hacer una inferencia es como sigue

$$\begin{aligned}a_t &= Ux_t + Ws_{t-1} + b_s, \\s_t &= \tanh(a_t), \\o_t &= c + Vs_t, \\y_t &= \text{softmax}(o_t)\end{aligned}$$

Más sobre inferencia en RNNs

$$\begin{aligned}a_t &= Ux_t + Ws_{t-1} + b_s, \\s_t &= \tanh(a_t), \\o_t &= c + Vs_t, \\y_t &= \text{softmax}(o_t)\end{aligned}$$

Lo que nos está diciendo esta expresión es que

- El valor del estado s en el instante t no solo depende de los pesos de los arcos que conectan las entradas con la capa oculta. Además, depende del resultado de modular el valor del estado anterior, s_{t-1} mediante la matriz W .
- Lo que antes era una sola multiplicación de matrices, ahora es una multiplicación recurrente (i.e. depende de ella misma)
- Las matrices U , W y V tienen siempre el mismo tamaño y no cambian
- La salida y es una secuencia: cada x_t genera una predicción, $y_t \in R^k$, tal que $y_t = \text{softmax}(o_t)$



¿Cómo entrenamos una red de este tipo?

- Recordemos que una red MLP sin recurrencia se entrena por gradiente descendente

$$w^{t+1} = w^t - \eta \frac{d}{dw} L(f(x; w), y)$$

- ¿Podemos aplicar las mismas ideas a una red recurrente?
 - Al fin y al cabo, necesitamos una función de pérdida...

[nature](#) > [letters](#) > article

Letter | Published: 09 October 1986

Learning representations by back-propagating errors

[David E. Rumelhart](#), [Geoffrey E. Hinton](#) & [Ronald J. Williams](#)

[Nature](#) 323, 533–536 (1986) | [Cite this article](#)

158k Accesses | 17k Citations | 574 Altmetric | [Metrics](#)



RNNs: backpropagation through time BPTT

- Nuestro vector de parámetros θ es ahora una tripla de matrices
 - $\langle W, U, V \rangle$
- Un ejemplo va a tener ahora la forma

$$(\{a_1, a_2, \dots, a_\tau\}, \{y_1, y_2, \dots, y_\tau\})$$

- La función de pérdida, entonces es suma de pérdidas en las predicciones realizadas para cada una de los token a_i a la entrada.

$$L(\{a_1, a_2, \dots, a_\tau\}, \{y_1, y_2, \dots, y_\tau\}) = \sum_{t=1}^{\tau} L_i = \sum_{t=1}^{\tau} \log \hat{p}(y_t) | \{x_1, \dots, x_t\})$$



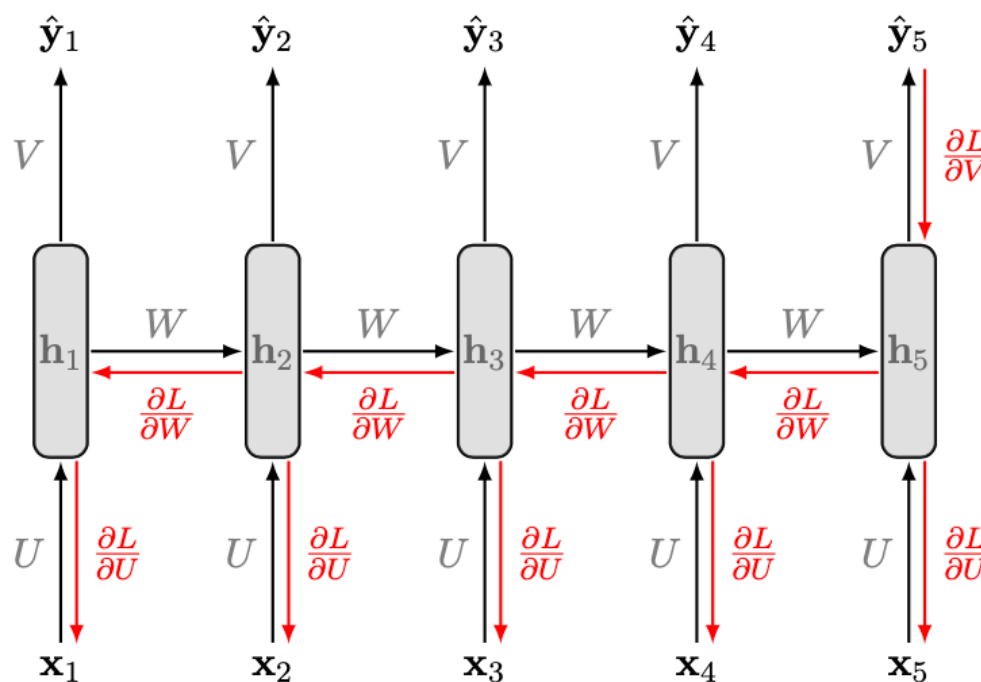
RNNs: backpropagation through time BPTT (y 2)

- Para computar L_t (la pérdida en el instante t , para el token t), necesitamos
 - Acceso a la capa oculta en el instante $t-1$
- La capa oculta en el instante t influye en el cálculo de y_t pero también en el cálculo de y_{t+1} y, por tanto, en L_{t+1}
- Por tanto, para obtener el error que podemos atribuir a s_t , necesitamos conocer su influencia en y_t , pero también en y_{t+1}
- Necesitamos un algoritmo de entrenamiento en dos pasadas
 - Primera pasada: hacia delante, calculamos los sucesivos s_t , y_t , cada vez, acumulando la pérdida obtenida en cada token y guardando s_t para usarlo en $t+1$
 - Segunda pasada: hacia atrás, procesamos la misma secuencia inversamente, calculando los gradients token a token, guardando los gradientes obtenidos en el instante t , para usarlos al calcular los del instante $t-1$

RNNs: backpropagation through time BPTT (y 3)



Máster en
Inteligencia
Artificial
UMU



Lecture 12
Recurrent Neural Networks II
CMSC 35246: Deep Learning

Shubhendu Trivedi
&
Rishi Kondor
University of Chicago
May 03, 2017





¿Cómo usamos RNNs con lenguaje natural?



RNNs y el lenguaje

- Queremos modelar lenguaje, es decir queremos estimar funciones como

$$P(\textit{fish}|\textit{Thanks for all the})$$

- De tal forma que podamos trabajar con cadenas según

$$P(w_{1:n}) = \prod_{i=1}^n P(w_i|w_{<i})$$

- Lo **distintivo** de las RNNs es que para predecir el token n-ésimo, no necesitamos todos los tokens desde n-1 hasta 1
 - Asumimos que h_{t-1} modela toda la información necesaria sobre el contexto pasado



¿Cómo codificamos las palabras para que las pueda usar una RNN?



RNNs y el lenguaje (y 2)

Sea la cadena de entrada $X=[x_1, \dots, x_t, \dots, x_N]$

- Cada x_i es un vector one-hot de tamaño $|V| \times 1$ (V es el tamaño del diccionario)
- Ese vector se codifica otra vez en su correspondiente embedding del token, y eso es lo que multiplica a U
- Luego se combina con la salida de la capa oculta para el estado anterior (s_{t-1}) modulada por W y esto genera la nueva salida s_t de la capa oculta (recursivamente será usada en la siguiente inferencia)
- El softmax genera una distribución de probabilidad sobre todo el vocabulario V

$$\begin{aligned}a_t &= Ux_t + Ws_{t-1} + b_s, \\s_t &= \tanh(a_t), \\o_t &= c + Vs_t, \\y_t &= \text{softmax}(o_t)\end{aligned}$$



Entrenamiento de RNNs como modelos de lenguaje

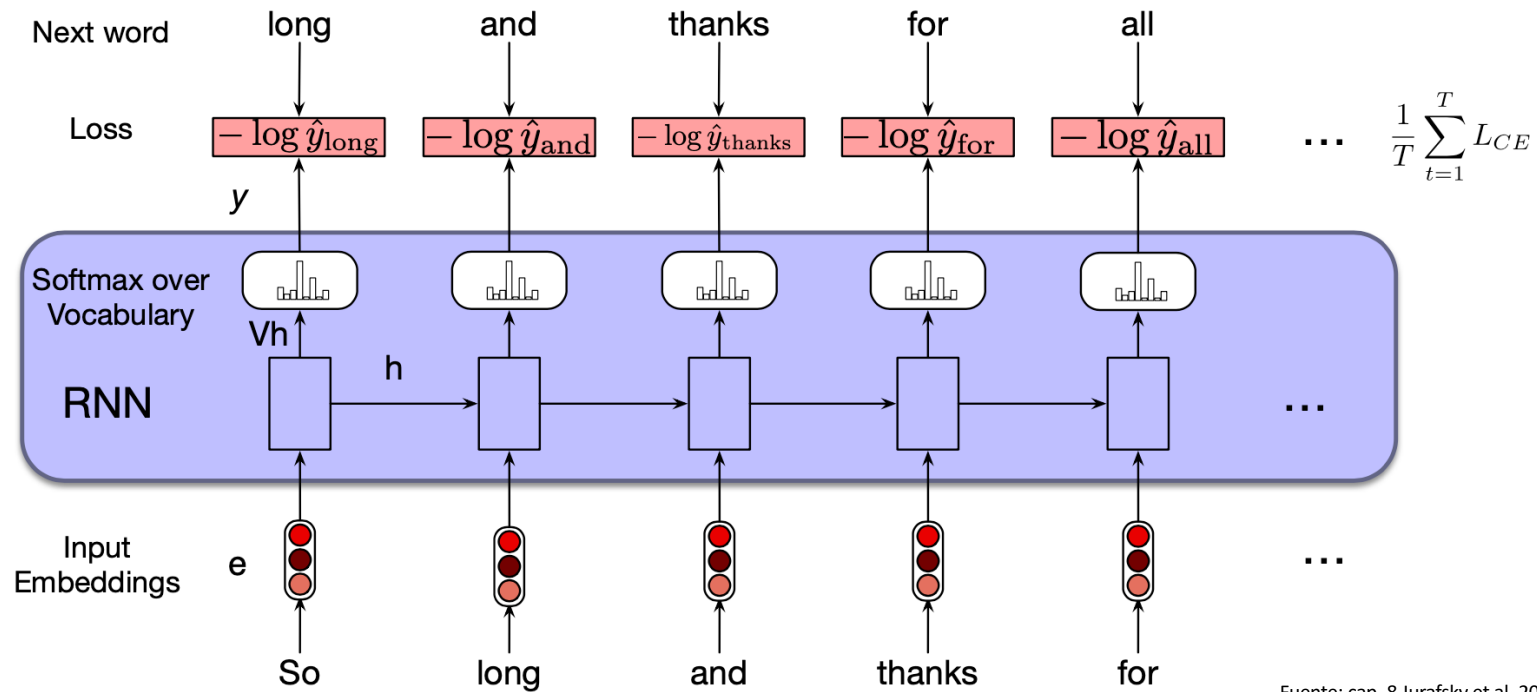
- Nos basamos en la auto-supervision
 - Corpus de texto como training data
 - Usamos fragmentos para predecir, en cada instante de tiempo qué token es el siguiente
 - Usamos entropía cruzada como función de pérdida

$$L_{CE} = - \sum_{w \in V} \mathbf{y}_t[w] \log \hat{\mathbf{y}}_t[w]$$

Siendo V el vocabulario, w una palabra del vocabulario, $\mathbf{y}_t[w]$ el componente del vector, correspondiente a w , de la salida softmax en el instante t



Entrenamiento de RNNs como modelos de lenguaje



Fuente: cap. 8 Jurafsky et al. 2024



Este es el bloque básico de construcción ¿cómo podemos aplicarlo a las tareas típicas de NLP?



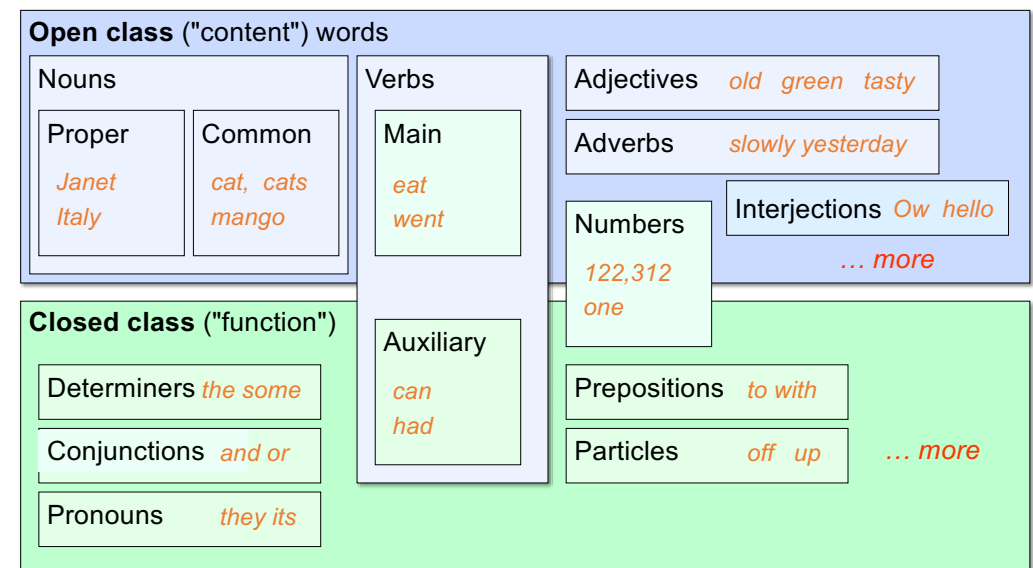
Este es el bloque básico de construcción ¿cómo podemos aplicarlo a las tareas típicas de NLP?

Etiquetado de secuencias

Etiquetado de secuencias

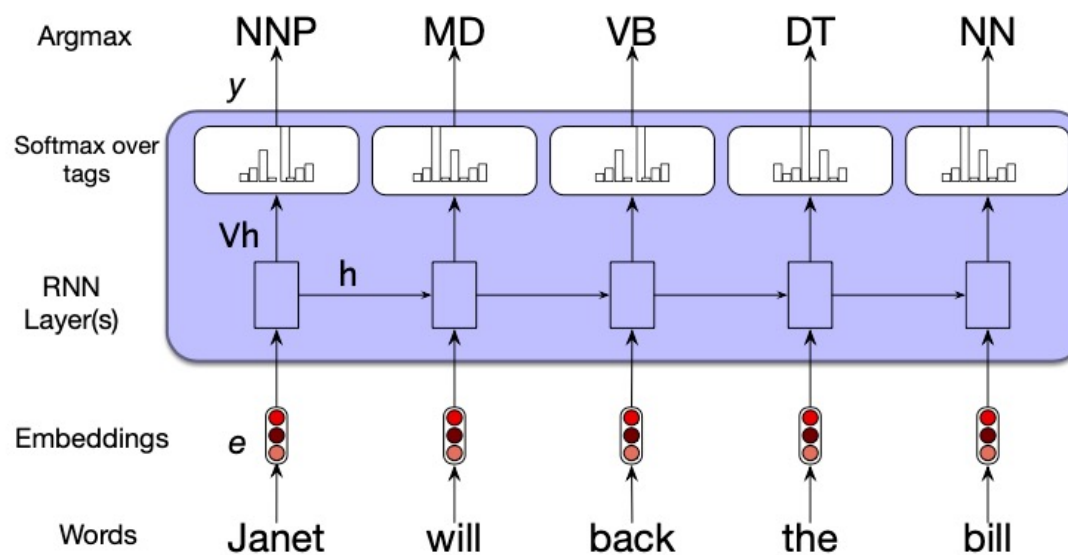


- Las palabras se clasifican en categorías (POS tags, parts of speech)
- En el etiquetado de secuencias usamos la RNN para etiquetar cada símbolo de la secuencia con una etiqueta. En realidad, se genera una distribución de probabilidad (softmax) para todas las tags
- Esta softmax es procesada posteriormente para generar la tag más probable en cada caso



Fuente: cap. 17 Jurafsky et al. 2024

Etiquetado de secuencias (y 2)

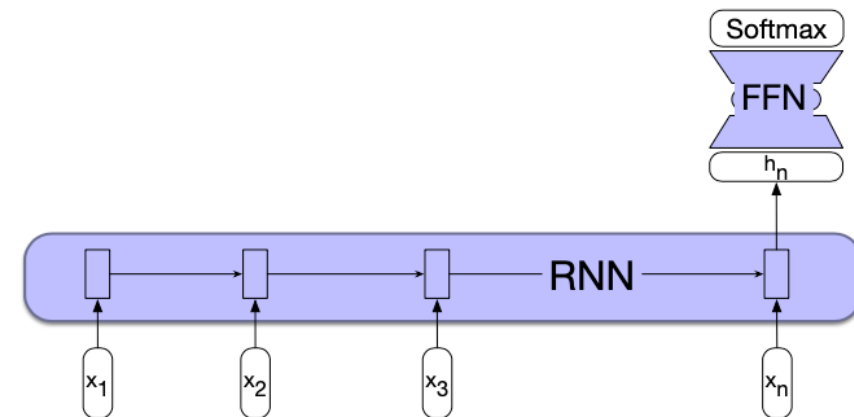


Fuente: cap. 8 Jurafsky et al. 2024



Clasificación de secuencias

- También denominado clasificación de textos (e.g., detección de spam, análisis de sentimientos, clasificación de documentos)
- Como su nombre indica, asignamos una etiqueta a la cadena en su totalidad
- En este caso, el estado interno de la RNN acumula el contexto del documento en su totalidad, y el que se obtiene al final se suele usar en una red MLP para decidir la etiqueta.
 - La RNN actúa como un codificador del documento: el resultado codificado es h_n , para un documento de longitud n
 - Una arquitectura alternativa usaría un h_n promedio de todos los h_i , $1 \leq i \leq n-1$ anteriores

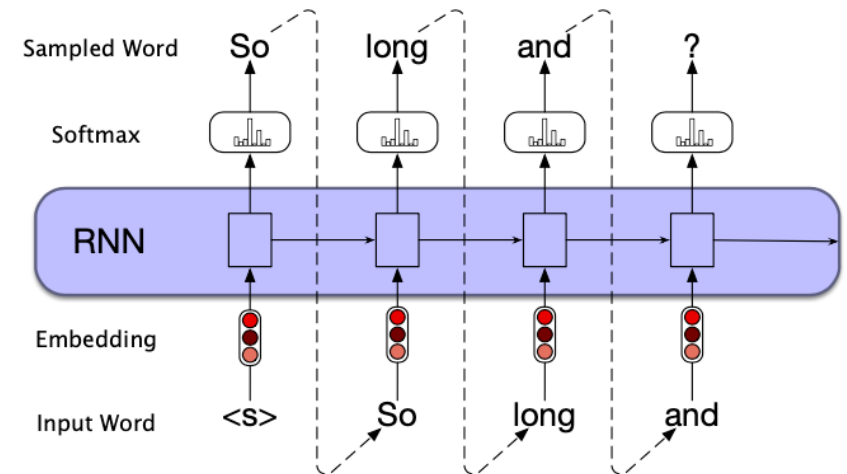


Fuente: cap. 8 Jurafsky et al. 2024



Generación de secuencias con RNNs

- Si hablamos de texto, responder preguntas, traducción, resumen de textos, corrección de gramática, generación de historias, diálogo interactivo, todas estas tareas necesitan de generación de texto
- Otros dominios incluyen diseño de proteínas, generación de datasets artificiales de procesos dependientes del tiempo para investigación, simulación y, por supuesto video!!
- Nos basamos en un enfoque autoregresivo: usamos la última salida del modelo (no el estado, la salida final!) para producir la siguiente
- Para producir la palabra en el instante t , usamos la obtenida en el instante $t-1$ como input. La del instante t se obtiene con un proceso aleatorio usando como base la distribución de probabilidad generada por la RNN
- El contexto inicial es importante (puede ser una frase, incluso un texto entero)



Fuente: cap. 8 Jurafsky et al. 2024



La naturaleza del aprendizaje en RNNs

- Las MLPs no tienen estado, las RNNs sí
 - Inferencia en MLP equivale a un servicio web sin estado (llamada GET en HTTP)
 - Inferencia en RNNs equivale a una serie de llamadas en una sesión Web
- Un MLP $f()$ procesa todo de una vez, la RNN $f'()$ es más compleja, necesita considerar el contexto de la cadena

$$f(x_t, \theta), f'(x_t, h_{t-1}, \theta)$$

- A la RNN se le pide dos cosas a la vez
 - Proporcionar un estado actual h_t suficientemente preciso
 - Actualizar la información para que los estados subsiguientes sigan siendo precisos $h_{t+1}, h_{t+2}, \dots$

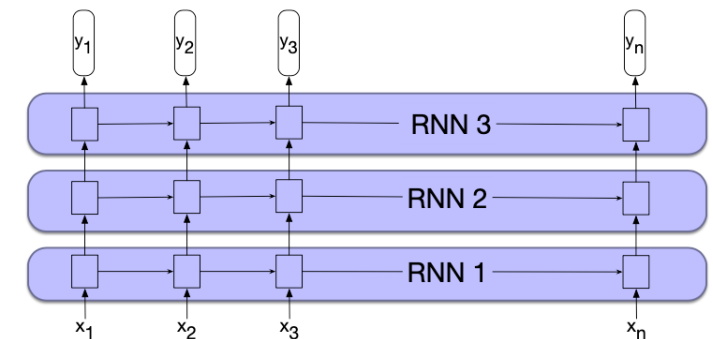


La naturaleza del aprendizaje en RNNs

- Veamos el ejemplo

Los vuelos que la aerolínea estaba cancelando iban llenos

- La RNN va a asignar una probabilidad alta a “estaba”, tras haber generado “aerolínea”, pues concuerdan en número
- No está tan claro con “iban” y “llenos” pues están lejos de “vuelos”
- ¿Cómo mantener un contexto preciso sin olvidar información pasada que va a ser de utilidad en el futuro?
- Las RNNs clásicas tienden a olvidar lo que queda cuando la cadena es demasiado larga.
 - Podríamos pensar en añadir más capas a las RNNs, pero empíricamente se ha visto que esto no ayuda. Son de utilidad para ahondar en abstracciones del lenguaje más potentes y complejas, pero no más dilatadas en el tiempo



Fuente: cap. 8 Jurafsky et al. 2024

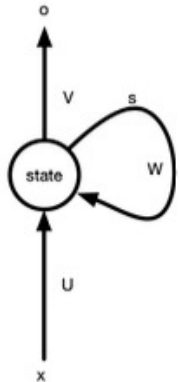
Vanishing Gradients!!! <https://ieeexplore.ieee.org/document/279181>



LSTMs (Long Short Term Memory)

(Hochreiter, Schmidhuber, 1997) <https://ieeexplore.ieee.org/abstract/document/6795963>

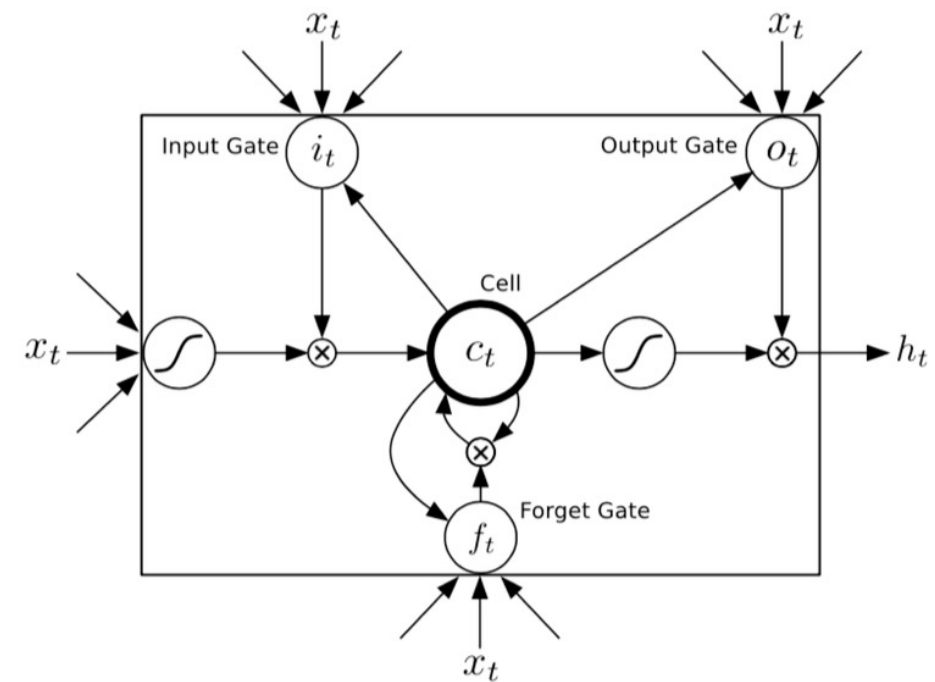
- El mecanismo de gestión del contexto en RNNs está codificado en la matriz de pesos W y dicha matriz siempre se aplica igual
 - Todo componente del contexto vale siempre lo que dicta W
 - ¿No podríamos aprender a flexibilizar esto según la entrada para determinar qué es útil en cada momento?
 - ¿qué información añadimos al contexto que pueda ser útil? ¿qué información eliminamos del contexto pues ya no nos va a servir?, de todo lo que disponemos, ¿qué usamos ahora mismo?
 - Todo esto se aprende, mediante sus correspondiente matrices de pesos articuladas en capas feedforward y salidas sigmoidales (saturación a 0/1)
- La LSTM añade una capa nueva para gestionar esto, y tres mecanismos (puertas lógicas) que abren y cierran el paso de información al contexto





Nodo de una LSTM

- Todas las puertas siguen el mismo patrón de diseño
 - Aceptan el token a la entrada
 - Basadas en una capa feedforward
 - Salida sigmoidal
 - La salida sigmoidal genera saturaciones a 0 o 1 (deja pasar o impide salida)
 - Cada puerta actúa mediante el elemento de información (entrada, qué olvidar o qué generar) que dejan pasar o impiden salida mediante un producto interno (x)
- El nodo cell es el que gestiona el contexto interno. Toda flecha que sale de él, comunica c_t al nodo destino





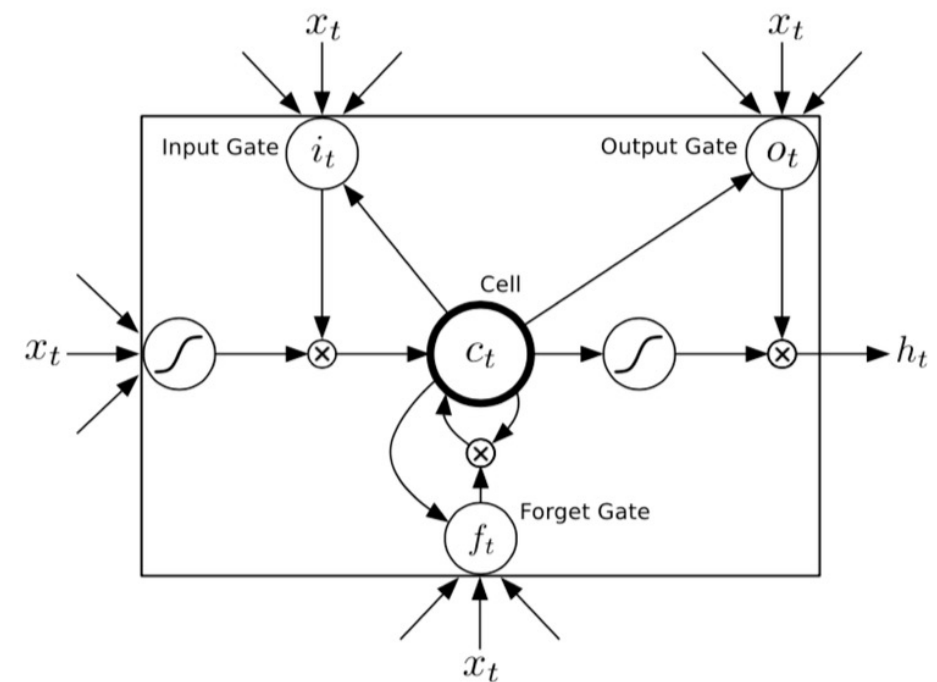
Puerta de olvido, forget gate

- Toma como entradas h_{t-1} (desde cell) y x_t (desde la entrada)
- Elimina información de c_t que ya no se necesita

$$\mathbf{f}_t = \sigma(\mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{W}_f \mathbf{x}_t)$$

$$\mathbf{k}_t = \mathbf{c}_{t-1} \odot \mathbf{f}_t$$

- El producto para obtener k_t es componente a componente
- Tenemos entonces las matrices U_f y W_f para capa oculta y entrada respectivamente
- Más tarde veremos cómo se usa k_t





Puerta de entrada

- La primera función es la típica de una RNN, actualizar el contexto con la entrada actual

$$\mathbf{g}_t = \tanh(\mathbf{U}_g \mathbf{h}_{t-1} + \mathbf{W}_g \mathbf{x}_t)$$

- La segunda es ver qué añadir al contexto de lo que acabamos de extraer

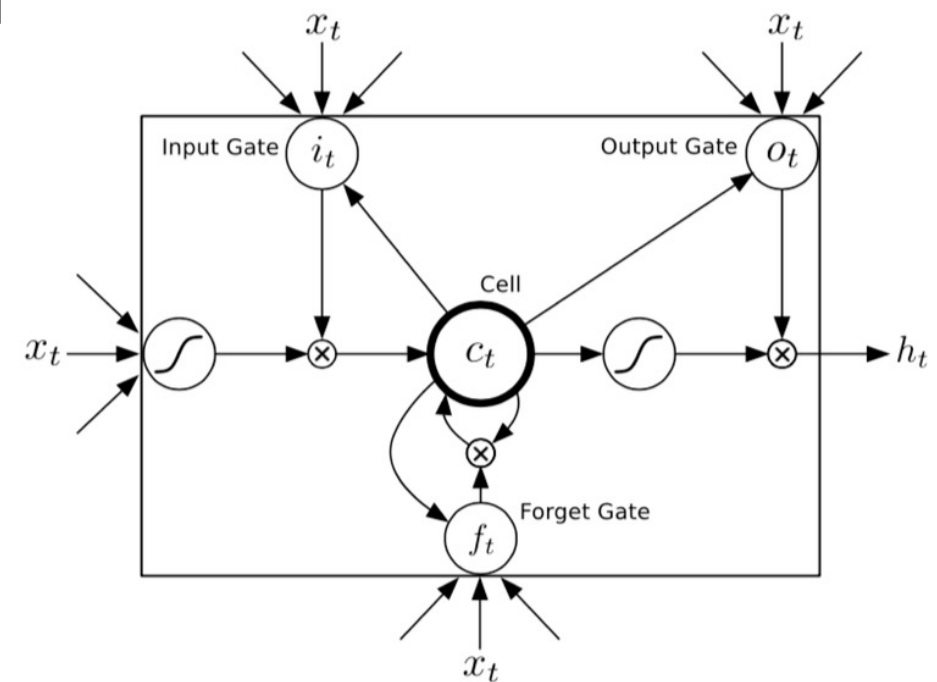
$$\mathbf{i}_t = \sigma(\mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{W}_i \mathbf{x}_t)$$

$$\mathbf{j}_t = \mathbf{g}_t \odot \mathbf{i}_t$$

- Finalmente, ya podemos obtener el contexto final $\mathbf{c}_t$ en el instante t

$$\mathbf{c}_t = \mathbf{j}_t + \mathbf{k}_t$$

- Como vemos, necesitamos aquí cuatro matrices más, $\mathbf{U}_g$, $\mathbf{W}_g$ y $\mathbf{U}_i$, $\mathbf{W}_i$ para gestionar capa oculta y entradas.





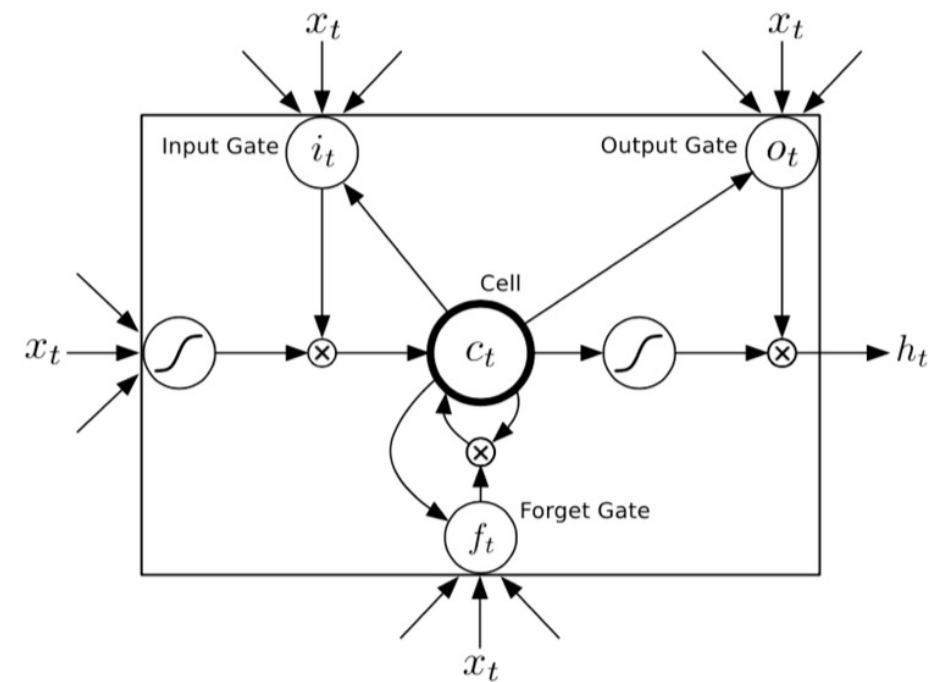
Puerta de salida

- En la puerta de salida decidimos qué información se requiere, del contexto actual, para generar el estado oculto actual

$$\mathbf{o}_t = \sigma(\mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{W}_o \mathbf{x}_t)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

- Y necesitamos dos nuevas matrices, $\mathbf{U}_o$, $\mathbf{W}_o$
- Nótese la diferencia entre la sigmoide (salida 0/1) y la tanh (salida 1/-1)
- Las puertas usan 0/1, para abrir y cerrar. Las operaciones típicas de una RRN usan tanh





Recapitulando

- Las ecuaciones son

$$\mathbf{f}_t = \sigma(\mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{W}_f \mathbf{x}_t)$$

$$\mathbf{k}_t = \mathbf{c}_{t-1} \odot \mathbf{f}_t$$

$$\mathbf{g}_t = \tanh(\mathbf{U}_g \mathbf{h}_{t-1} + \mathbf{W}_g \mathbf{x}_t)$$

$$\mathbf{i}_t = \sigma(\mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{W}_i \mathbf{x}_t)$$

$$\mathbf{j}_t = \mathbf{g}_t \odot \mathbf{i}_t$$

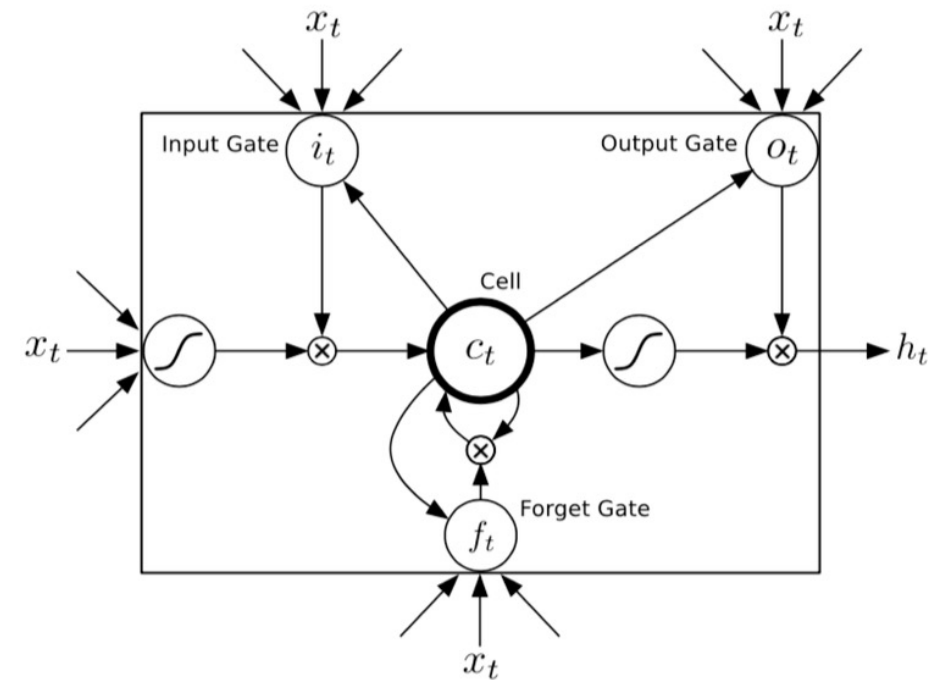
$$\mathbf{c}_t = \mathbf{j}_t + \mathbf{k}_t$$

$$\mathbf{o}_t = \sigma(\mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{W}_o \mathbf{x}_t)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

- Y las matrices por tanto

- Para la entrada $\mathbf{W}_f, \mathbf{W}_g, \mathbf{W}_i, \mathbf{W}_o$
- Para el estado oculto $\mathbf{U}_f, \mathbf{U}_g, \mathbf{U}_i, \mathbf{U}_o$





Referencias

- El libro de Jurafsky, referencia principal de la asignatura,
<https://web.stanford.edu/~jurafsky/slp3/>,
capítulos 8 y algo del 5 y 17
- Para profundizar más sobre el BPTT
(Backpropagation through time), ir al capítulo 9 de
Dive Into Deep Learning

https://d2l.ai/chapter_recurrent-neural-networks/bptt.html

